

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks:30

Annexure A

Reverse Engineering Task

Code of Conduct:

I E.Ragul / 192571032 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E. Ragul

Annexure A

Reverse Engineering Task

1. Legacy E-Learning Management System

An existing E-Learning Management System has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

Question:

Reverse engineer the existing system by identifying appropriate classes, responsibilities, attributes, methods, and relationships. Redesign the system using object-oriented principles and layered architecture, and explain how the redesigned structure improves cohesion, coupling, maintainability, and scalability.

ANSWER:

1. Reverse Engineering of the Legacy System

The existing E-Learning Management System can be analyzed by identifying the major objects involved in each module.

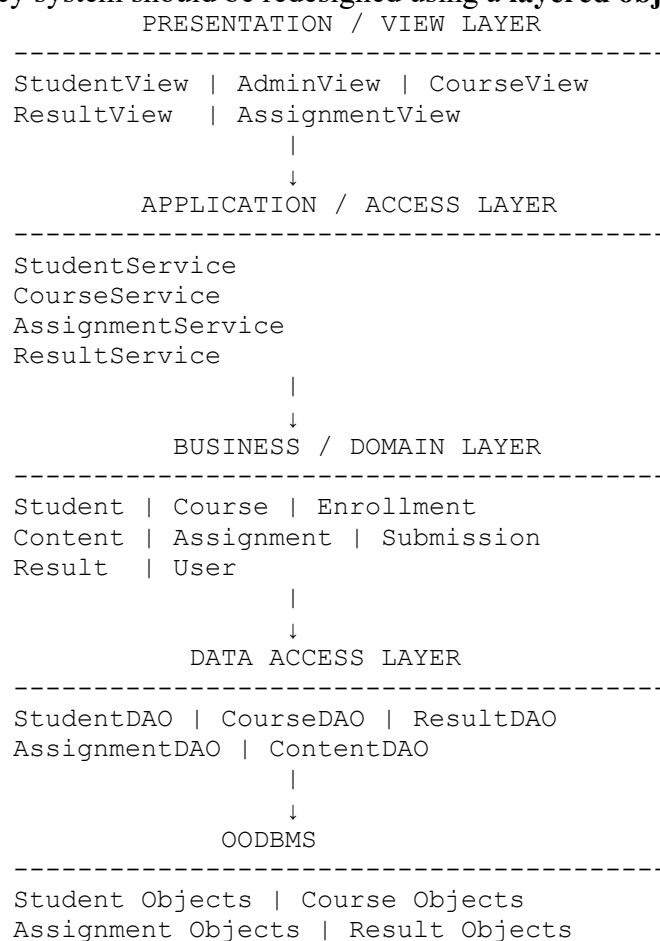
Class	Main Attributes	Main Methods	Responsibility
Student	studentId, name, email, password	register(), login(), enrollCourse()	Manage student information and enrollment
Course	courseId, title, description, duration	createCourse(), updateCourse(), addContent()	Manage course details
Content	contentId, type, title, fileUrl	upload(), view(), download()	Manage learning materials
Assignment	assignmentId, title, deadline, marks	create(), submit(), evaluate()	Manage assignments
Submission	submissionId, submittedDate, answer, marks	submit(), grade()	Store student submissions
Enrollment	enrollmentId, enrollmentDate, status	enroll(), cancel()	Connect students with courses
Result	resultId, marks, grade	calculateGrade(), generateResult()	Generate student results
Admin	adminId, name, email	manageCourse(), manageStudent()	Manage system operations
User	userId, name, email, password	login(), logout()	Common user functionality

Main Relationships

- **Student** → **Enrollment**: A student can enroll in many courses.
- **Course** → **Enrollment**: A course can have many students.
- **Course** → **Content**: A course contains multiple learning contents.
- **Course** → **Assignment**: A course can contain multiple assignments.
- **Student** → **Submission**: A student can submit assignments.
- **Assignment** → **Submission**: An assignment can have many submissions.
- **Student** → **Result**: A student can have multiple results.
- **User** → **Student/Admin**: Student and Admin inherit common user properties.

2. Redesigned Object-Oriented Architecture

The legacy system should be redesigned using a **layered object-oriented architecture**.



3. Layer Responsibilities

A. Presentation / View Layer

This layer handles interaction with users.

Classes:

- StudentView
- AdminView
- CourseView
- AssignmentView
- ResultView

Responsibilities:

- Display course information.
- Accept student input.
- Display assignment and result information.
- Send requests to the application layer.
- Avoid implementing business rules.

For example:

StudentView → StudentService → Student

The view does not directly access the database.

B. Application / Access Layer

This layer acts as the **access layer** between the view and business objects.

Classes:

- StudentService
- CourseService
- AssignmentService
- ResultService

Example:

```
class CourseService
{
public:
    void registerStudent();
    void enrollStudent();
    void createCourse();
    void updateCourse();
};
```

Responsibilities:

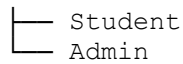
- Coordinate operations.
- Validate requests.
- Call appropriate domain objects.
- Control transactions.
- Communicate with DAOs.
- Keep the presentation layer independent from the database.

C. Business / Domain Layer

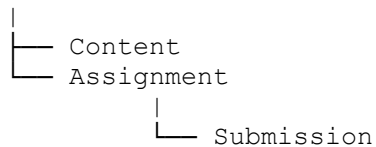
This is the core of the system.

Important domain classes include:

User



Student — Enrollment — Course



Student ————— Result

The domain layer contains actual business rules.

For example:

Student → enrollCourse()

Course → addContent()

Assignment → submit()

Submission → evaluate()

Result → calculateGrade()

This layer does not depend on the user interface.

D. Data Access Layer

The Data Access Layer communicates with the **Object-Oriented Database Management System (OODBMS)**.

Example classes:

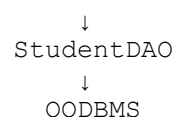
- StudentDAO
- CourseDAO
- ContentDAO
- AssignmentDAO
- ResultDAO

Example:

```
class StudentDAO
{
public:
    void save(Student student);
    Student findById(int id);
    void update(Student student);
    void remove(int id);
};
```

The DAO hides database operations from the business layer.

StudentService



Because an OODBMS stores objects directly, objects such as Student, Course, and Assignment can be persisted while maintaining their object relationships.

4. Object-Oriented Principles Used

Encapsulation

Data and methods are combined inside classes.

```
Student
├── studentId
├── name
├── email
└── enrollCourse()
```

Private attributes prevent unauthorized modification.

Inheritance

Common properties can be placed in `User`.

```
      User
     /   \
  Student Admin
```

This reduces duplicate code.

Polymorphism

Different users can implement common operations differently.

```
User
└── login()
```

`Student` → student-specific login behavior

`Admin` → administrator-specific login behavior

Abstraction

Complex database operations are hidden behind DAO interfaces.

`CourseService` → `CourseDAO` → `OODBMS`

The service does not need to know how objects are physically stored.

5. Design Patterns

1. MVC Pattern

The presentation architecture can use **Model-View-Controller**.

`View` → `Controller` → `Model`

```
      ↓
    Service
      ↓
      DAO
      ↓
    OODBMS
```

This separates user interface from system logic.

2. DAO Pattern

The **Data Access Object** pattern isolates database operations.

CourseService



CourseDAO



OODBMS

If the database implementation changes, the business layer requires minimal modification.

3. Factory Pattern

A **UserFactory** can create different user objects.

UserFactory



— Student

— Admin

4. Observer Pattern

Students can receive notifications when course content, assignments, or results change.

Course



Notification System



Students

6. Example System Flow

When a student enrolls in a course:

StudentView



StudentController



StudentService



Student / Course



Enrollment



EnrollmentDAO



OODBMS

The result is returned in the reverse direction:

OODBMS



ResultDAO



ResultService



ResultView



Student

Thus, the **access and view layers are integrated with the OODBMS through the business and data-access layers**, rather than directly accessing the database.

7. Improvements in the Redesigned System

Quality	Legacy System	Redesigned System
Cohesion	One module performs many tasks	Each class/layer has a focused responsibility
Coupling	UI tightly coupled with business logic	Layers communicate through defined interfaces
Maintainability	Changes can affect many modules	Changes are localized to specific layers
Scalability	Difficult to add features	New services/classes can be added independently
Reusability	Limited	Domain classes and services can be reused
Testability	Difficult to test	Each layer can be tested independently
Database independence	UI/business directly depends on DB	DAO isolates OODBMS operations
Security	Database may be directly accessed by UI	Access is controlled through service/DAO layers

Conclusion

The legacy E-Learning Management System can be successfully reverse engineered into classes such as **Student, Course, Content, Assignment, Submission, Enrollment, and Result**. A **layered object-oriented architecture** separates the View, Access/Application, Business/Domain, and Data Access responsibilities. Encapsulation, inheritance, abstraction, and polymorphism improve the object design, while **MVC, DAO, Factory, and Observer patterns** provide structured solutions to common design problems.

The redesigned system achieves **high cohesion and low coupling**, making it easier to maintain, test, modify, and scale. The **OODBMS is accessed through the Data Access Layer**, while the View Layer communicates through application services, preventing direct database dependency and creating a cleaner, more maintainable architecture.

2. Legacy Customer Support System

A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

Question:

Analyze the existing implementation and identify its object-oriented design limitations. Reverse engineer and refactor the system using suitable behavioral design patterns, and explain how the redesigned solution improves flexibility, extensibility, loose coupling, and maintainability.

ANSWER:

1. Analysis of Existing System

The legacy Customer Support System uses many **if-else or switch statements** to decide how a customer request should be handled.

Example:

```
if channel == "Email"
    handleEmail()

else if channel == "Chatbot"
    handleChatbot()

else if channel == "Telephone"
    handleTelephone()

else if channel == "SocialMedia"
    handleSocialMedia()
```

When a new channel such as **WhatsApp** is added, several existing modules may need to be modified.

2. Object-Oriented Design Limitations

1. High Coupling

The main support module directly depends on every support channel.

```
SupportSystem
├── Email
├── Chatbot
├── Telephone
└── SocialMedia
```

Changing one channel can affect the main system.

2. Low Cohesion

One class is responsible for:

- Identifying the channel
- Selecting the handler

3. Difficult to Extend

Adding a new channel requires modifying existing conditional statements.

For example:

```
if Email
else if Chatbot
else if Telephone
else if SocialMedia
else if WhatsApp ← New modification
```

4. Violates Open/Closed Principle

The system must be **modified** whenever a new channel is introduced instead of simply extending it.

5. Poor Maintainability

As the number of channels increases, the conditional statements become larger and harder to understand.

3. Reverse Engineering – Identified Classes

The existing system can be decomposed into the following classes:

Class	Attributes	Methods	Responsibility
SupportRequest	requestId, customerId, message, priority	createRequest(), getDetails()	Store support request
Customer	customerId, name, contact	createRequest(), viewResponse()	Represents customer
SupportHandler	handlerId, name	handleRequest()	Common support handling interface
EmailHandler	emailAddress	handleRequest()	Process email requests
ChatbotHandler	botId	handleRequest()	Process chatbot requests
TelephoneHandler	phoneNumber	handleRequest()	Process telephone requests
SocialMediaHandler	accountId	handleRequest()	Process social-media requests
SupportManager	requestQueue	assignRequest()	Coordinate requests

4. Suitable Behavioral Design Pattern

The most suitable pattern is the **Chain of Responsibility Pattern**.

It allows a support request to pass through a chain of handlers until the appropriate handler processes it.

```
Customer Request
    ↓
Email Handler
    ↓
Chatbot Handler
```

↓
Telephone Handler
↓
Social Media Handler

5. Chain of Responsibility Design

Create a common interface:

```
class SupportHandler
{
protected:
    SupportHandler* nextHandler;

public:
    SupportHandler()
    {
        nextHandler = nullptr;
    }

    void setNext(SupportHandler* next)
    {
        nextHandler = next;
    }

    virtual void handleRequest(string channel) = 0;
};
```

Email Handler

```
class EmailHandler : public SupportHandler
{
public:
    void handleRequest(string channel)
    {
        if(channel == "Email")
            cout << "Email request handled";
        else if(nextHandler)
            nextHandler->handleRequest(channel);
    }
};
```

Chatbot Handler

```
class ChatbotHandler : public SupportHandler
{
public:
    void handleRequest(string channel)
    {
        if(channel == "Chatbot")
            cout << "Chatbot request handled";
        else if(nextHandler)
            nextHandler->handleRequest(channel);
    }
};
```

Telephone Handler

```
class TelephoneHandler : public SupportHandler
{
public:
    void handleRequest(string channel)
    {
        if(channel == "Telephone")
            cout << "Telephone request handled";
        else if(nextHandler)
            nextHandler->handleRequest(channel);
    }
};
```

Social Media Handler

```
class SocialMediaHandler : public SupportHandler
{
public:
    void handleRequest(string channel)
    {
        if(channel == "SocialMedia")
            cout << "Social media request handled";
        else if(nextHandler)
            nextHandler->handleRequest(channel);
    }
};
```

6. Creating the Handler Chain

```
EmailHandler
    ↓
ChatbotHandler
    ↓
TelephoneHandler
    ↓
SocialMediaHandler
```

The system can then send a request to the first handler:

```
email.setNext(&chatbot);
chatbot.setNext(&telephone);
telephone.setNext(&socialMedia);
```

```
email.handleRequest("Telephone");
```

The request moves through the chain until `TelephoneHandler` processes it.

7. Adding a New Support Channel

Suppose **WhatsApp** support is introduced.

Create a new class:

```
class WhatsAppHandler : public SupportHandler
{
public:
    void handleRequest(string channel)
    {
        if(channel == "WhatsApp")
```

```

        cout << "WhatsApp request handled";
    else if (nextHandler)
        nextHandler->handleRequest(channel);
    }
};

```

The existing handlers do not need to be rewritten.

Email



Chatbot



Telephone



Social Media



WhatsApp

This demonstrates the **Open/Closed Principle**: the system is open for extension but closed for unnecessary modification.

8. Other Behavioral Pattern That Can Be Used

Strategy Pattern

The **Strategy Pattern** can also be used when the system knows which channel should handle the request and wants to select the handling algorithm dynamically.

SupportService



SupportStrategy



Email Chatbot Phone SocialMedia

Each strategy implements:

handleRequest()

The support service can change the strategy without changing its main logic.

Observer Pattern

The **Observer Pattern** can be used for notifications.

SupportRequest

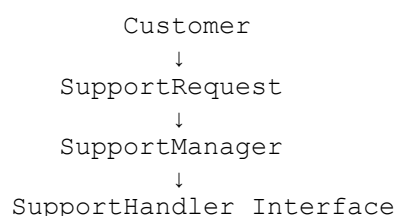


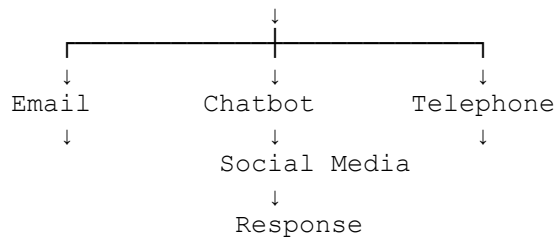
Observers



Email SMS Dashboard

9. Redesigned Architecture





The **Chain of Responsibility** separates request processing from the object that initiates the request.

10. Improvements

Aspect	Legacy System	Redesigned System
Flexibility	Fixed conditional statements	Handlers can be rearranged or replaced
Extensibility	Existing modules must be modified	New handler can be added
Coupling	High coupling	Low coupling through common interface
Maintainability	Large conditional logic	Small independent classes
Reusability	Limited	Handlers can be reused
Testing	Difficult	Each handler can be tested independently
Scalability	Becomes complex with more channels	Easily supports additional channels
OCP	Violated	Better compliance with Open/Closed Principle

Conclusion

The legacy Customer Support System suffers from **high coupling, low cohesion, large conditional statements, poor extensibility, and violation of the Open/Closed Principle**. By reverse engineering the system into independent support-handler classes and applying the **Chain of Responsibility behavioral pattern**, each channel becomes responsible for its own request processing.

The redesigned system can easily support **email, chatbot, telephone, social media, WhatsApp, or future channels** without changing the core support manager. Therefore, the design provides **greater flexibility, extensibility, loose coupling, scalability, and maintainability**.

3. Legacy Banking Transaction System

A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

Question:

Reverse engineer the system by identifying appropriate domain classes, service classes, and data access components. Redesign the application using suitable layers and design patterns, and explain how the redesigned architecture improves separation of concerns, testability, and system maintainability.

ANSWER:

1. Analysis of the Existing System

The legacy banking application contains **account management, transaction validation, fund transfer, loan processing, reporting, and database operations** inside a few tightly coupled classes.

A typical legacy design may look like:

```
BankingSystem
├── Account Management
├── Transaction Validation
├── Fund Transfer
├── Loan Processing
├── Report Generation
└── Database Operations
```

For example, a fund-transfer method may directly:

```
Validate account
    ↓
Check balance
    ↓
Update sender
    ↓
Update receiver
    ↓
Save to database
    ↓
Generate transaction record
```

This creates several design problems.

2. Reverse Engineering – Domain Classes

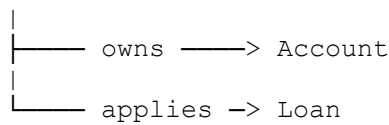
The business objects can be separated into independent domain classes.

Domain Class	Attributes	Main Methods	Responsibility
Customer	customerId, name, address, phone	updateDetails()	Store customer information
Account	accountNo, balance, accountType, status	deposit(), withdraw(), getBalance()	Manage account
Transaction	transactionId, date,	create(), updateStatus()	Represent a banking

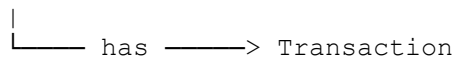
	amount, type, status		transaction
FundTransfer	transferId, fromAccount, toAccount, amount	transfer()	Represent fund transfer
Loan	loanId, amount, interestRate, duration, status	applyLoan(), calculateInterest()	Manage loan information
LoanApplication	applicationId, customerId, amount, status	submit(), approve(), reject()	Handle loan requests
TransactionReport	reportId, dateRange, transactions	generateReport()	Represent transaction reports

Relationships

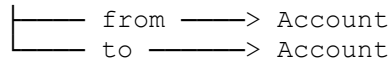
Customer



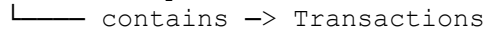
Account



FundTransfer



TransactionReport



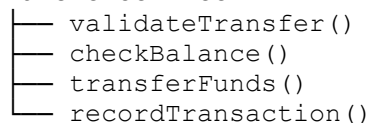
3. Service Layer

Business operations should be moved into separate **service classes**.

Service Class	Responsibility
AccountService	Create, update, and manage accounts
TransactionService	Validate and process transactions
TransferService	Perform fund transfers
LoanService	Process loan applications
ReportService	Generate transaction reports

For example:

TransferService



The service classes contain business logic but **do not directly perform database operations**.

4. Data Access Layer

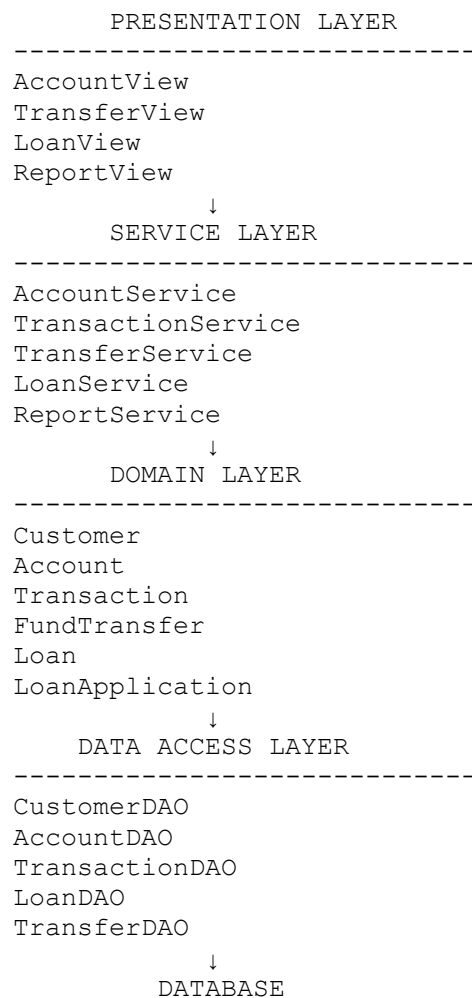
Database operations should be removed from business methods and placed inside **DAO (Data Access Object)** classes.

```
AccountDAO
TransactionDAO
CustomerDAO
LoanDAO
TransferDAO
```

Example:

```
class AccountDAO
{
public:
    Account findById(int accountNo);
    void save(Account account);
    void update(Account account);
};
```

5. Redesigned Layered Architecture

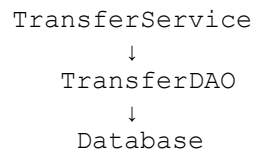


This provides a clear separation between **user interface**, **business logic**, **domain objects**, and **data storage**.

6. Design Patterns Used

A. DAO Pattern

The **DAO Pattern** isolates database operations.



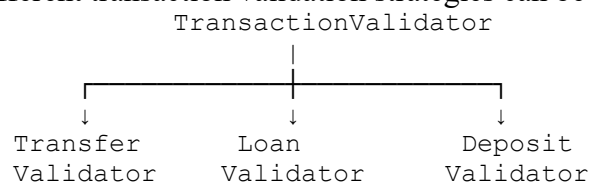
The service does not need to know whether the data is stored in an SQL database, OODBMS, or another persistence system.

Benefit

- Reduces database dependency.
- Improves testability.
- Makes database changes easier.
- Keeps business classes clean.

B. Strategy Pattern

Different transaction validation strategies can be implemented separately.



Each strategy implements its own validation rules.

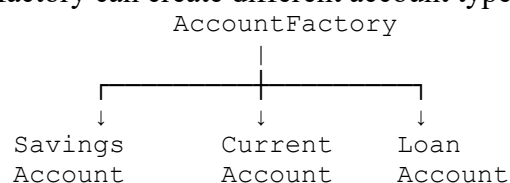
For example:

```
class TransactionStrategy
{
public:
    virtual bool validate() = 0;
};
```

A new validation strategy can be added without modifying the existing transaction service.

C. Factory Pattern

A factory can create different account types.



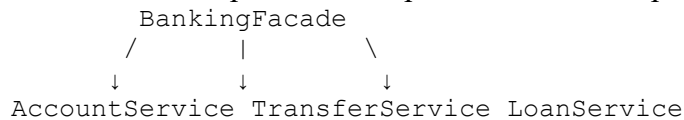
Example:

```
Account* account = AccountFactory::createAccount("Savings");
```

This centralizes object creation.

D. Facade Pattern

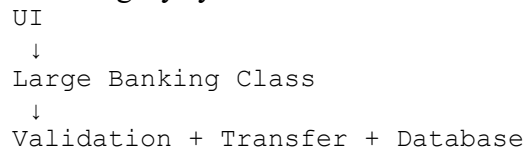
A `BankingFacade` can provide a simple interface to the presentation layer.



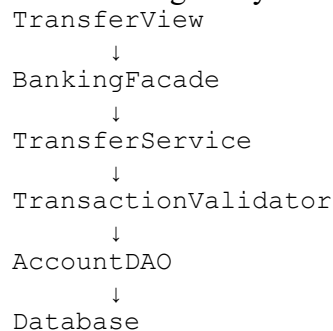
The view does not need to communicate with multiple services directly.

7. Example Fund Transfer Flow

In the legacy system:



In the redesigned system:



Step-by-step

1. `TransferView` receives sender, receiver, and amount.
2. `BankingFacade` forwards the request.
3. `TransferService` coordinates the operation.
4. `TransactionValidator` checks the transaction.
5. `AccountDAO` retrieves account objects.
6. `TransferService` performs the business operation.
7. `AccountDAO` persists the updated accounts.
8. `TransactionDAO` records the transaction.
9. The result is returned to the view.

8. Separation of Concerns

Each layer has one primary responsibility.

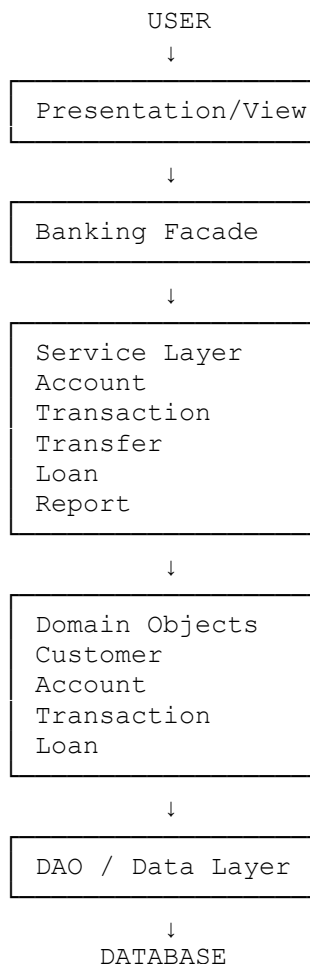
Presentation	→ What the user sees
Service	→ What the system does
Domain	→ What the banking objects represent
DAO	→ How data is stored/retrieved
Database	→ Where data is stored

For example, the `TransferService` should **not** contain SQL/database code, and `AccountDAO` should **not** contain loan approval rules.

9. Improvements

Aspect	Legacy System	Redesigned System
Separation of concerns	Responsibilities mixed together	Each layer has a specific responsibility
Coupling	High	Low
Testability	Difficult because database is embedded in business logic	Services and DAOs can be tested separately
Maintainability	Changes affect multiple modules	Changes are localized
Reusability	Business logic tied to UI/database	Services can be reused
Scalability	Difficult to add banking features	New services and domain classes can be added
Database dependency	Business methods directly access database	DAO isolates database operations
Object creation	Scattered throughout system	Factory centralizes creation

10. Final Architecture



Conclusion

The legacy banking system suffers from **tight coupling, mixed responsibilities, embedded database operations, and poor testability**. Reverse engineering identifies separate domain objects such as **Customer, Account, Transaction, FundTransfer, Loan, and LoanApplication**.

The redesigned system uses a **layered architecture** with Presentation, Service, Domain, and Data Access layers. The **DAO, Strategy, Factory, and Facade patterns** further separate responsibilities and reduce dependencies.

As a result, the system becomes **easier to test, maintain, extend, and scale**. Database changes can be handled within the data-access layer, while banking rules remain inside the service/domain layers, providing a cleaner and more robust banking application.

4. Legacy Hotel Reservation System

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

Question:

Analyze the existing architecture and identify problems related to coupling, duplication, and persistence management. Reverse engineer the system by introducing suitable DAO, Repository, or Data Access patterns, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.

ANSWER:

1. Analysis of the Existing Architecture

In the legacy system, the **reservation screens directly communicate with the OODBMS**.

Reservation Screen

↓
OODBMS

↓
Room / Guest / Booking / Payment

The same database operations are repeated in different modules.

For example:

Reservation Module → find available rooms

Booking Module → find available rooms

Payment Module → find booking

Guest Module → find guest

This creates several design problems.

2. Problems in the Existing System

A. High Coupling

The user interface is directly dependent on the OODBMS.

ReservationScreen

↓
OODBMS Queries

If the database structure or persistence mechanism changes, many screens must be modified.

B. Code Duplication

The same queries are written repeatedly.

Room query → Reservation module

Room query → Booking module

Room query → Admin module

This increases code size and maintenance effort.

C. Poor Persistence Management

Database operations are mixed with presentation logic.

For example:

Screen Code

- Display room
- Search room
- OODBMS query
- Save booking
- Payment operation

The screen should not be responsible for persistence.

D. Difficult Testing

UI modules cannot easily be tested independently because they directly depend on the OODBMS.

E. Poor Reusability

A database operation written inside one screen cannot easily be reused by another module.

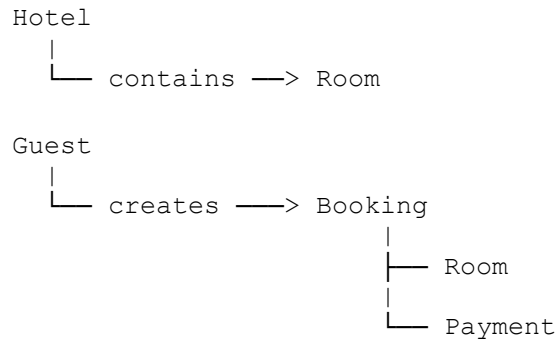
3. Reverse Engineering – Domain Classes

The major domain objects can be identified as follows:

Class	Attributes	Main Methods	Responsibility
Guest	guestId, name, email, phone	register(), updateDetails()	Manage guest information
Room	roomNo, type, price, status	checkAvailability(), updateStatus()	Manage room information
Booking	bookingId, guestId, roomNo, checkIn, checkOut	createBooking(), cancelBooking()	Manage reservations
Payment	paymentId, bookingId, amount,	makePayment(), refund()	Manage payments

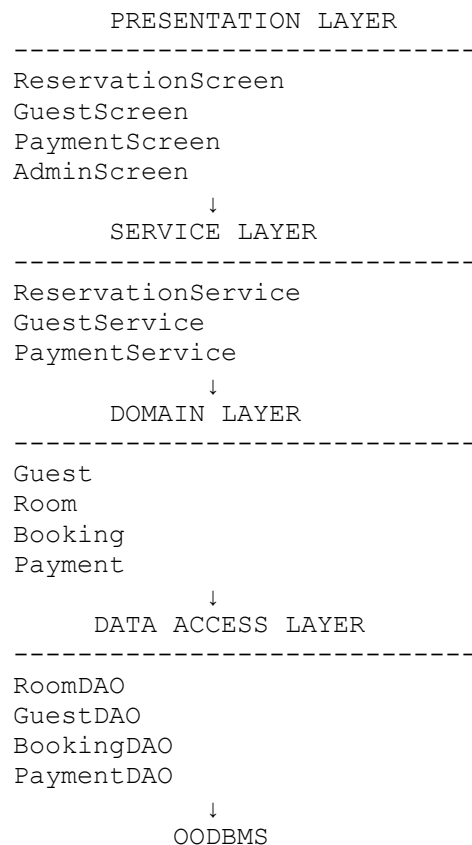
	method, status		
Hotel	hotelId, name, address	addRoom(), removeRoom()	Manage hotel information

Relationships



4. Redesigned Architecture

The system should introduce a **Data Access Layer** between the application and the OODBMS.



This removes direct OODBMS access from the reservation screens.

5. DAO Pattern

The **Data Access Object (DAO) Pattern** provides a separate class for database operations.

RoomDAO

```
class RoomDAO
{
public:
    Room findById(int roomNo);
    vector<Room> findAvailableRooms();
    void save(Room room);
    void update(Room room);
};
```

GuestDAO

```
class GuestDAO
{
public:
    Guest findById(int guestId);
    void save(Guest guest);
    void update(Guest guest);
};
```

BookingDAO

```
class BookingDAO
{
public:
    Booking findById(int bookingId);
    void save(Booking booking);
    void update(Booking booking);
    void remove(int bookingId);
};
```

PaymentDAO

```
class PaymentDAO
{
public:
    Payment findById(int paymentId);
    void save(Payment payment);
    void update(Payment payment);
};
```

Now database operations are centralized.

6. Repository Pattern

A **Repository** provides a collection-like interface for domain objects.

For example:

```
RoomRepository
├─ findById()
├─ findAvailable()
├─ add()
```



```

BookingRepository
├── findById()
├── findByGuest()
├── add()
└── remove()

```

The service layer works with repositories instead of directly accessing the OODBMS.

```

ReservationService

```

↓

```

RoomRepository

```

↓

```

OODBMS

```

The repository hides persistence details from the business logic.

7. DAO vs Repository

DAO	Repository
Focuses mainly on data-access operations	Focuses on domain objects
Closely related to persistence	Provides collection-like domain interface
Can contain database-specific operations	Hides persistence details from business logic
Useful for OODBMS persistence operations	Useful for domain-oriented application design

For this hotel system, **DAO can handle low-level OODBMS operations**, while **Repository can provide a clean domain-oriented interface**.

They can also be used together:

```

ReservationService

```

↓

```

RoomRepository

```

↓

```

RoomDAO

```

↓

```

OODBMS

```

8. OODBMS Integration

Because the system uses an **Object-Oriented DBMS**, objects such as `Room`, `Guest`, and `Booking` can be persisted as objects.

The DAO manages operations such as:

```

Create Object

```

↓

```

Store Object in OODBMS

```

```

Retrieve Object

```

↓

```

Return Domain Object

```

```

Update Object

```

↓

```

Persist Updated Object

```

```

Delete Object

```

↓

```

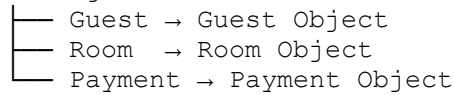
Remove Object

```

Object relationships can also be maintained.

For example:

Booking

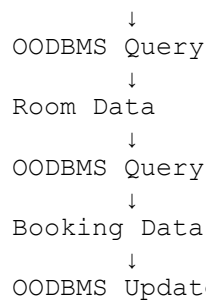


The application does not need to know the internal persistence mechanism.

9. Example Reservation Flow

Legacy System

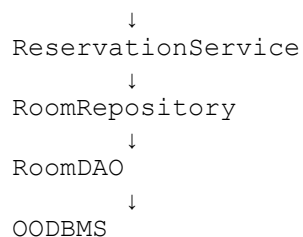
Reservation Screen



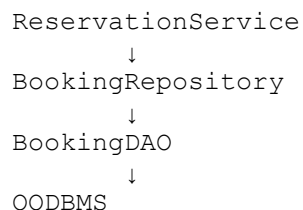
The screen contains both UI and persistence logic.

Redesigned System

ReservationScreen



Then:

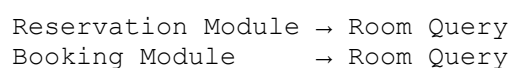


This keeps the screen completely independent of database operations.

10. Removing Duplication

Suppose three modules need available rooms.

Legacy



Admin Module → Room Query

Redesigned

Reservation Module

 \
Booking Module ----> RoomRepository → RoomDAO → OODBMS
 /

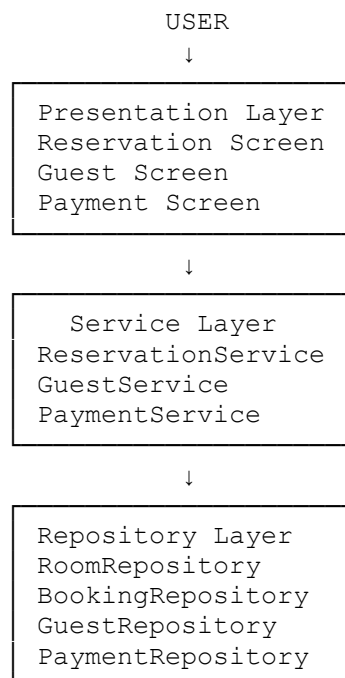
Admin Module

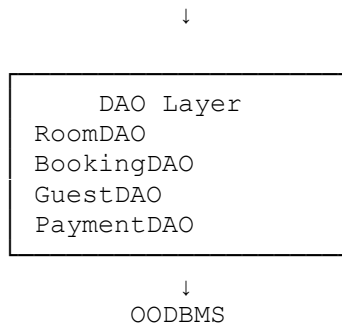
The room-search operation is implemented **once** and reused by multiple modules.

11. Improvements

Aspect	Legacy System	Redesigned System
Coupling	UI tightly coupled to OODBMS	UI independent of OODBMS
Duplication	Queries repeated	Centralized DAO/Repository
Persistence	Mixed with UI code	Dedicated data-access layer
Modularity	Low	High
Reusability	Limited	High
Testing	Difficult	Easier with mocked DAOs/repositories
Maintainability	Database changes affect many modules	Changes isolated to data layer
OODBMS integration	Direct and scattered	Centralized and controlled
Scalability	Difficult	Easier to extend

12. Final Architecture





Conclusion

The legacy Hotel Reservation System suffers from **high coupling, duplicated database queries, poor persistence management, and weak modularity** because reservation screens directly access the OODBMS.

The redesigned system introduces **DAO and Repository patterns** to separate persistence from business and presentation logic. The **Service Layer** handles reservation rules, the **Repository Layer** provides reusable domain-oriented operations, and the **DAO Layer** manages OODBMS persistence.

As a result, the system achieves **low coupling, reduced duplication, high modularity, better reusability, easier testing, and cleaner OODBMS integration**. Adding new modules such as online check-in or room-service booking can be done without duplicating database operations.

5. Legacy Transportation Management System

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

Question:

Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable creational design patterns, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.

ANSWER:

1. Analysis of the Existing System

The legacy transportation application has separate modules for:

- Bus
- Train
- Taxi
- Rental Vehicle

Object creation logic is repeated in many places.

For example:

```
if type == "Bus"
    create Bus

else if type == "Train"
    create Train

else if type == "Taxi"
    create Taxi

else if type == "Rental"
    create RentalVehicle
```

The same logic may appear in booking, payment, scheduling, and management modules. This causes **duplication, tight coupling, and difficult maintenance**.

2. Object-Oriented Design Limitations

A. Duplicate Object Creation

Different modules independently create transportation objects.

```
Booking Module    → creates Bus
Schedule Module   → creates Bus
Payment Module    → creates Bus
```

The same creation logic is repeated.

B. High Coupling

Application classes directly depend on concrete classes.

```
Booking
├── Bus
├── Train
├── Taxi
└── RentalVehicle
```

Adding a new type requires changing multiple classes.

C. Difficult Extensibility

If `Metro` is introduced, several modules may need new conditions.

```
if Bus
else if Train
else if Taxi
else if Rental
else if Metro      ← new modification
```

D. Violation of Open/Closed Principle

Existing code must repeatedly be modified when new transportation types are introduced.

3. Reverse Engineering – Common Abstraction

All transportation types share common characteristics.

Therefore, create an abstract base class:

```
class Transport
{
protected:
    string id;
    string name;
    double fare;

public:
    virtual void display() = 0;
    virtual void calculateFare() = 0;
    virtual ~Transport() {}
};
```

This provides a common abstraction.

4. Concrete Transportation Classes

Bus

```
class Bus : public Transport
{
public:
    void display()
    {
        cout << "Bus Transport";
    }

    void calculateFare()
    {
        cout << "Calculating Bus Fare";
    }
};
```

Train

```
class Train : public Transport
{
public:
    void display()
    {
        cout << "Train Transport";
    }

    void calculateFare()
    {
        cout << "Calculating Train Fare";
    }
};
```

Taxi

```
class Taxi : public Transport
{
public:
    void display()
    {
        cout << "Taxi Transport";
    }

    void calculateFare()
    {
        cout << "Calculating Taxi Fare";
    }
};
```

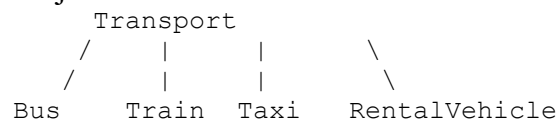
Rental Vehicle

```
class RentalVehicle : public Transport
{
public:
    void display()
    {
        cout << "Rental Vehicle";
    }

    void calculateFare()
    {
        cout << "Calculating Rental Fare";
    }
};
```

5. Object Relationships

The redesigned object model is:



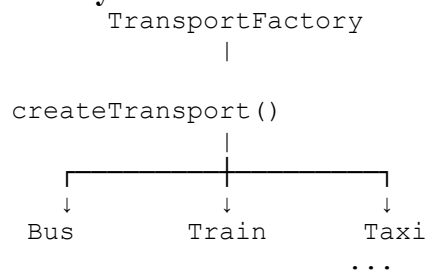
Other classes can work with the abstract `Transport` type instead of depending on each concrete class.

```
Booking
  ↓
Transport
  ↓
Bus / Train / Taxi / RentalVehicle
```

This provides **polymorphism**.

6. Factory Method Pattern

The **Factory Method Pattern** moves object creation into a dedicated factory.



Example:

```
class TransportFactory
{
public:
    static Transport* createTransport(string type)
    {
        if(type == "Bus")
            return new Bus();

        else if(type == "Train")
            return new Train();

        else if(type == "Taxi")
            return new Taxi();

        else if(type == "Rental")
            return new RentalVehicle();

        return nullptr;
    }
};
```

The client now uses:

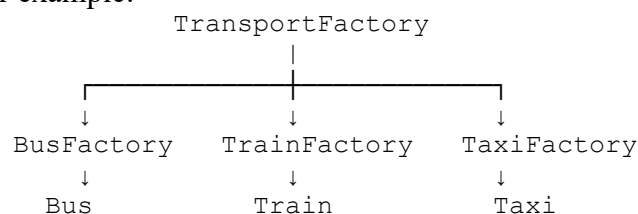
```
Transport* t = TransportFactory::createTransport("Bus");
t->display();
```

The client does not directly create `Bus`, `Train`, or `Taxi`.

7. Abstract Factory Pattern

If the system needs **families of related transportation objects**, Abstract Factory is more suitable.

For example:



A common factory interface can be defined:

```
class TransportFactory
{
public:
    virtual Transport* createTransport() = 0;
};
```


Then:

```
class BusFactory : public TransportFactory
{
public:
    Transport* createTransport()
    {
        return new Bus();
    }
};
```

Similarly:

TrainFactory → Train

TaxiFactory → Taxi

RentalFactory → RentalVehicle

8. When to Use Each Pattern

Factory Method	Abstract Factory
Creates one type of product	Creates families of related products
Simpler transportation system	Useful for larger transportation systems
Easy to implement	More flexible for multiple product families
Suitable for Bus/Train/Taxi creation	Suitable when each transport has related services

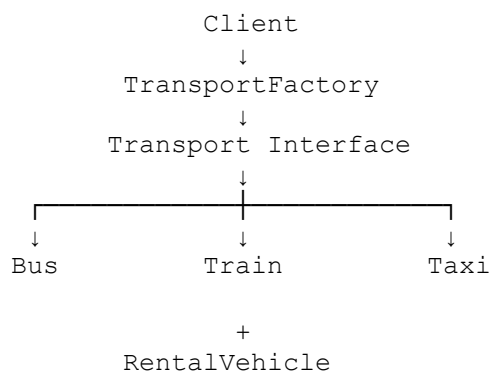
For this system, **Factory Method is sufficient for basic transportation creation**. Abstract Factory can be introduced if the system later needs families such as:

Bus + BusTicket + BusSchedule

Train + TrainTicket + TrainSchedule

Taxi + TaxiBooking + TaxiFare

9. Improved Architecture



The booking system interacts with:

Transport

rather than:

Bus

Train

Taxi

RentalVehicle

directly.

10. Adding a New Transportation Service

Suppose the company introduces **Metro**.

Legacy System

Multiple modules must be changed:

Booking Module
Schedule Module
Payment Module
Management Module

Each may need:

```
else if type == "Metro"
```

Factory Design

Create:

```
class Metro : public Transport
{
public:
    void display()
    {
        cout << "Metro Transport";
    }

    void calculateFare()
    {
        cout << "Calculating Metro Fare";
    }
};
```

Then update the factory:

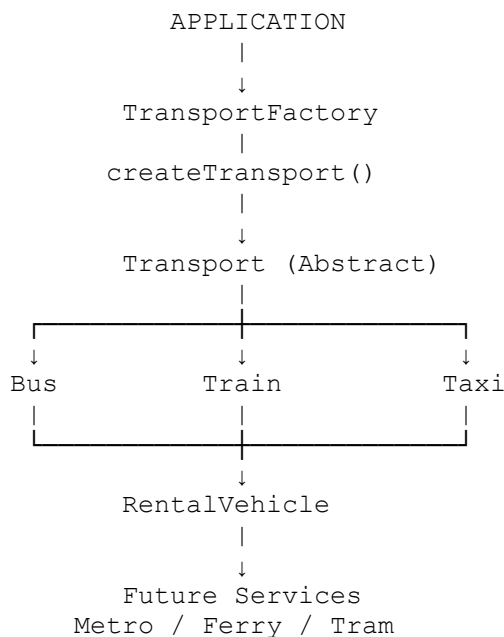
```
TransportFactory
    ↓
createTransport("Metro")
    ↓
Metro
```

The rest of the application can continue working with `Transport`.

11. Benefits of the Redesigned System

Aspect	Legacy System	Factory-Based Design
Object creation	Repeated in many modules	Centralized
Coupling	High	Low
Code reuse	Low	High
Extensibility	Difficult	Easy
Maintainability	Difficult	Easier
Scalability	Adding services affects many modules	New classes can be added
Polymorphism	Limited	Strong
Open/Closed Principle	Frequently violated	Better supported

12. Final Architecture



Conclusion

The legacy Transportation Management System suffers from **duplicated object creation, tight coupling, poor scalability, and difficult maintenance**. Reverse engineering reveals a common **Transport** abstraction with concrete classes such as **Bus, Train, Taxi, and RentalVehicle**.

The **Factory Method Pattern** centralizes object creation and allows the application to depend on the **Transport** abstraction rather than concrete classes. If the system needs families of related transportation products, the **Abstract Factory Pattern** can be used.

The redesigned system therefore provides **better code reuse, loose coupling, easier extension, improved scalability, and simpler maintenance**. New services such as **Metro, Ferry, or Tram** can be introduced with minimal impact on existing application code.

6. Legacy Healthcare Monitoring System

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

Question:

Reverse engineer the application by identifying violations of SOLID principles, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

ANSWER:

6. Legacy Healthcare Monitoring System

1. Analysis of the Legacy System

The existing Healthcare Monitoring System performs many unrelated operations inside a few large classes.

For example:

```
HealthcareSystem
├── Patient management
├── Sensor reading
├── Alert generation
├── Medical record storage
└── Report generation
```

The classes also directly depend on concrete devices and storage implementations.

```
MonitoringSystem
```

↓

```
ConcreteSensor
```

↓

```
ConcreteDatabase
```

This produces high coupling and makes the system difficult to modify and test.

2. Reverse Engineering – Main Classes

The system can be decomposed into the following classes:

Class	Attributes	Methods	Responsibility
Patient	patientId, name, age	updateDetails(), getDetails()	Store patient information
SensorReading	sensorId, value, timestamp	getValue(), getTime()	Represent sensor data
MonitoringService	patient, sensors	collectReading(), processReading()	Coordinate monitoring
AlertService	alertType, message	generateAlert()	Generate alerts
MedicalRecord	recordId, patientId, readings	addReading(), getRecords()	Manage medical records
ReportService	reportData	generateReport()	Generate reports
Sensor	sensorId, type	readValue()	Common sensor abstraction
RecordRepository	—	save(), find()	Store and retrieve records

3. SOLID Violations

A. Single Responsibility Principle — SRP

SRP states that a class should have one main responsibility.

The legacy `HealthcareSystem` may perform:

```
HealthcareSystem
├── Read sensor
├── Process patient data
├── Generate alert
├── Save medical record
└── Generate report
```

This gives the class multiple reasons to change.

Redesign

Separate responsibilities:

```
MonitoringService
AlertService
RecordService
ReportService
```

Each class now has a focused responsibility.

4. Open/Closed Principle — OCP

OCP states that software should be open for extension but closed for unnecessary modification.

The legacy system may contain:

```
if sensor == "HeartRate"
    readHeartRate()

else if sensor == "Temperature"
    readTemperature()

else if sensor == "BloodPressure"
    readBloodPressure()
```

Adding a new sensor requires modifying existing code.

Redesign

Create a common interface:

```
class ISensor
{
public:
    virtual double readValue() = 0;
    virtual ~ISensor() {}
};
```

5. Dependency Inversion Principle — DIP

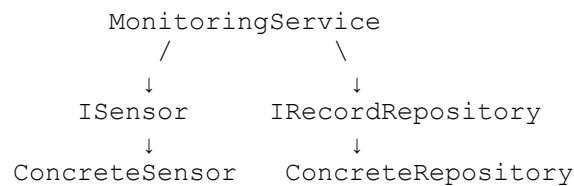
DIP states that high-level modules should depend on abstractions rather than concrete implementations.

Legacy Design

```
MonitoringService
  ↓
ConcreteHeartRateSensor
  ↓
ConcreteDatabase
```

The service is directly dependent on specific implementations.

Redesigned Design



The high-level service depends only on interfaces.

6. Sensor Abstraction

Define a common sensor interface:

```
class ISensor
{
public:
    virtual SensorReading read() = 0;
    virtual ~ISensor() {}
};
```

Different devices implement it:

```
class HeartRateSensor : public ISensor
{
public:
    SensorReading read()
    {
        // Read heart-rate data
    }
};
class TemperatureSensor : public ISensor
{
public:
    SensorReading read()
    {
        // Read temperature data
    }
};
```

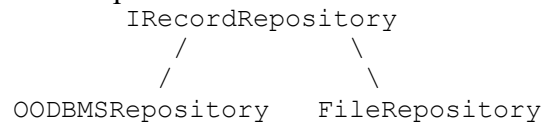
The monitoring service doesn't need to know the concrete sensor type.

7. Storage Abstraction

The same approach should be applied to medical-record storage.

```
class IRecordRepository
{
public:
    virtual void save(MedicalRecord record) = 0;
    virtual MedicalRecord find(int id) = 0;
    virtual ~IRecordRepository() {}
};
```

Possible implementations:



The application can use an OODBMS repository without embedding database-specific operations inside business classes.

8. Redesigned Service Classes

MonitoringService

```
class MonitoringService
{
private:
    ISensor* sensor;
    IRecordRepository* repository;

public:
    MonitoringService(ISensor* s,
                     IRecordRepository* r)
    {
        sensor = s;
        repository = r;
    }

    void monitor()
    {
        SensorReading reading = sensor->read();

        // Process reading
        // Save medical record
    }
};
```

The service depends on **interfaces**, not concrete sensor or database classes.

9. Alert Service

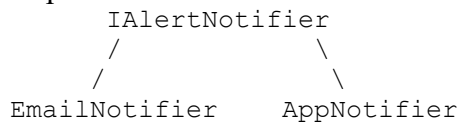
Alert processing should also be separated.

```
class AlertService
{
public:
    void checkReading(SensorReading reading)
    {
        // Apply alert rules
        // Generate notification when required
    }
};
```

For multiple alert mechanisms, an interface can be introduced:

```
class IAlertNotifier
{
public:
    virtual void sendAlert(string message) = 0;
    virtual ~IAlertNotifier() {}
};
```

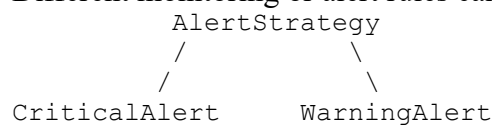
Possible implementations:



10. Suitable Design Patterns

A. Strategy Pattern

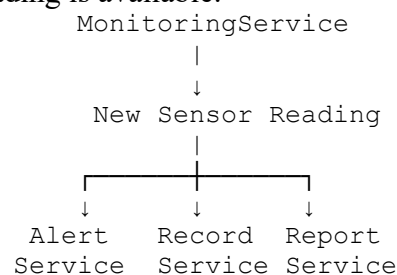
Different monitoring or alert rules can be represented as strategies.



This allows alert rules to be changed without modifying the monitoring service.

B. Observer Pattern

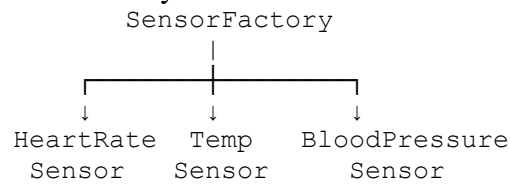
The **Observer Pattern** is suitable for notifying interested components when a new sensor reading is available.



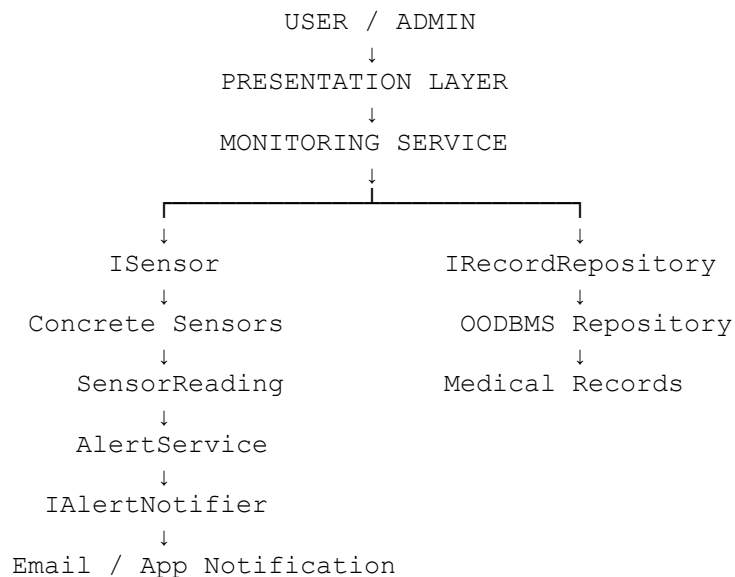
When a reading is generated, registered observers can respond automatically.

C. Factory Pattern

A sensor factory can create different sensors.



11. Final Architecture



12. SOLID Improvements

Principle	Legacy System	Redesigned System
SRP	Classes perform many unrelated tasks	Each class has one responsibility
OCP	New sensors require modifying existing logic	New sensors implement <code>ISensor</code>
DIP	Services depend on concrete classes	Services depend on interfaces
Cohesion	Low	High
Coupling	High	Low
Flexibility	Difficult to replace devices/storage	Implementations can be replaced
Testability	Difficult because of concrete dependencies	Interfaces can be mocked
Maintainability	Changes affect many classes	Changes are localized

13. Example of Improved Testability

A fake sensor can be used during testing:

```
class FakeSensor : public ISensor
{
public:
    SensorReading read()
    {
        // Return test reading
    }
};
```

The monitoring service can then be tested without connecting to a real monitoring device.

Similarly, a fake repository can be used:

MonitoringService

↓
FakeSensor
+

FakeRepository

This makes unit testing much easier.

14. Overall Benefits

Cohesion

Each class performs a focused task.

Loose Coupling

Services communicate through interfaces rather than concrete implementations.

Flexibility

Sensors, repositories, and notification mechanisms can be replaced easily.

Testability

Mock or fake implementations can be injected during testing.

Maintainability

Changes to one component have minimal impact on other components.

Scalability

New sensors, storage mechanisms, alert types, and reporting systems can be added without redesigning the entire application.

Conclusion

The legacy Healthcare Monitoring System violates **SRP, OCP, and DIP** because large classes perform multiple unrelated responsibilities and depend directly on concrete sensors and storage components.

The redesigned architecture separates the system into **domain objects, monitoring services, alert services, repositories, and interfaces**. The **Strategy, Observer, and Factory** patterns further improve flexibility and object creation.

By making high-level services depend on abstractions such as `ISensor`, `IRecordRepository`, and `IAAlertNotifier`, the system achieves **high cohesion, loose coupling, better testability, flexibility, maintainability, and scalability**.

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation